Mapping Best Empirical Models

A Meta-Analysis of Winning Solutions in Kaggle Tabular Competitions

Kenneth Young · CS495 Capstone in Data Science, Bellevue College

June 2026 · Instructor: Pedro Albuquerque, PhD

Contents

Abstract

What distinguishes a winning tabular machine-learning solution from a *near*-winning one? This study addresses that question through a meta-analysis of 45 top-three solutions from Kaggle tabular competitions concluded between February 2022 and April 2026 — predominantly the synthetic-data Playground Series — each coded across a structured field schema, a qualitative attribution pass, and a 53-technique feature-engineering taxonomy, and compared against 11 near-winning (rank 4–25) controls from the same competitions.

Three patterns stand out. First, winning solutions reduce to a small set of recurring paradigms, dominated by ensemble stacking (28 of 45 entries). Second, winning feature engineering is a modest, shared core — target encoding plus a few interaction and encoding techniques — atop a long bespoke tail; the typical winner applies only a handful of their own techniques (a median of four among the best-documented entries), and this already-modest volume does not cleanly separate winners from near-winners: the comparison is framing-sensitive and underpowered (11 paired competitions), with only learned/advanced features leaning toward winners, on small counts. Third, the documented top is a small recurring guild: 5 of 11 near-winning controls share an author with a winners corpus that spans only 34 distinct authors, and a few central figures supply widely-adopted techniques without themselves winning.

An intra-rater blind re-code of a 12-entry subsample showed substantial agreement at the technique-family level (Cohen's $\kappa = 0.65$; lower at the individual-technique level). The study contributes a reusable feature-engineering taxonomy and a replicable, audited multi-pass coding workflow. All findings are bounded by a documentation-self-selection limitation and the largely synthetic-data setting — the corpus is the documented elite, not the typical competitor or real-world tabular practice. Among that elite, winning appears to turn less on the volume of feature engineering than

on paradigm fit, a recurrent core of techniques, and membership in a small recurring guild — as much about who competes as about technique.

---

1. Introduction

1.1 Background and Context

A machine learning competition is a structured contest in which participants submit predictive models against a shared dataset, with submissions ranked by a stated evaluation metric. The Kaggle platform (kaggle.com) hosts the largest public collection of such competitions and a community of several million participants. Competitions span domains from medical diagnostics to retail forecasting, but the dominant data modality on the platform is *tabular* — structured rows-and-columns data of the kind generated by databases, sensors, and business processes, and the setting in which practitioner folklore most strongly credits feature engineering as the decisive craft (the claim this study tests).

Two competition formats matter here. **Featured competitions** are monetized, sponsor-funded contests with real-world prize pools, often run on proprietary real-world datasets supplied by the sponsoring organization. **Playground Series** competitions — the source of nearly all data in this study — are non-monetized monthly contests run by Kaggle itself on *synthetic* datasets generated from publicly available real-world datasets. This synthetic origin is consequential: because the data come from a known generative process applied to a recoverable source, some winning strategies exploit properties of the *generator* rather than modeling signal — a regime that does not arise in most real-world tabular problems, and one that bounds how far these findings generalize (Sections 3.2, 5.4). The Tabular Playground Series (TPS), which ran from 2021 to mid-2022, was the precursor format, later folded into the renamed Playground Series in 2023.

Winning solutions on Kaggle are typically documented in three places: a *writeup* posted to the competition's discussion forum after the contest closes, one or more *notebooks* (executable code, sometimes published on Kaggle itself and sometimes on the author's GitHub), and the *leaderboard* (which records final scores and rankings). The writeup is the primary source of qualitative information about which techniques were used and why; the notebook provides the verifiable code-level pipeline; the leaderboard provides the score margin between the winner and the runners-up. Crucially, this documentation is voluntary and uneven: not every top finisher publishes a writeup, and those who do are not a random sample of winners — a selection effect this study must confront directly (Section 3.8).

1.2 Problem Statement

Despite the public availability of thousands of competition writeups, the machine learning literature contains no systematic empirical characterization of winning tabular Kaggle solutions as a population. Practitioner-facing knowledge about "what wins on Kaggle" is overwhelmingly anecdotal, conveyed through individual blog posts, tutorial notebooks, or conference talks by a small number of high-profile competitors. Academic benchmarking studies of tabular machine learning (Caruana et al.; Gorishniy et al.; Erickson et al.) compare *algorithms* on standardized datasets, not *strategies* on competitive submissions. This leaves two open questions. First, when authors actually win tabular competitions, what do they do — and, crucially, is it *distinctive of winning*, or merely shared with the broad field of strong competitors? Characterizing winners in isolation can-

not answer the second part: establishing what *distinguishes* a winning approach requires comparing winners against strong non-winners, which prior work has not done.

Second, and inseparable from the first, is the *social structure* of how winning techniques arise and propagate. Kaggle competitions are public-by-default events with extensive cross-referencing between authors, notebooks, and discussion threads, and techniques attributed to one prominent contributor often turn out to have been documented earlier by community members who never themselves win. If what distinguishes winning is not a private technical edge but participation in a small, connected community, then "what wins" becomes partly a question of *who competes* — yet the relationship between technique origination, attribution, and competitive outcome has not been studied systematically.

1.3 Research Questions

Building on the two gaps above — whether winning is technically distinctive, and whether it is socially structured — the analysis is organized around four questions (RQ1–RQ3 address the first; RQ4 the second):

- **RQ1.** What recurring strategy *paradigms* characterize winning tabular solutions, and how do those choices relate to the constraints of the competition? *(typology and coupling evidence — Sections 4.2–4.3)*
- **RQ2.** Which feature-engineering techniques do winners use, and how much? *(prevalence — Section 4.4)*
- **RQ3.** Does feature engineering distinguish winners from strong non-winners? *(winner–control comparison — Section 4.5)*
- **RQ4.** How is the winning population structured, and how does knowledge move through it? *(community and attribution — Section 4.6)*

1.4 Contributions

This work contributes:

1. a four-paradigm typology of winning tabular solutions, and a coupling-evidence framework for testing claimed constraint→strategy patterns — most of which the corpus does not support;
2. a 53-technique feature-engineering taxonomy (organized into 11 families) with substantial family-level inter-coder agreement ($\kappa = 0.65$) — a reusable, reliability-checked coding instrument;
3. the empirical finding that winning feature engineering is a small shared core (a median of four own techniques among well-documented winners), not a uniformly elaborate craft;
4. the finding that feature-engineering volume does not *cleanly* separate winners from near-winners (aggregate and within-competition paired comparisons disagree, both non-significant at n = 11), with learned/advanced features the one family more often present in winners (on small counts);
5. a characterization of the documented top as a small recurring guild (45% author overlap with the controls) with traceable knowledge propagation; and
6. the identification of a documentation-self-selection effect that bounds any writeup-based study of competition winners, and the explicit scoping of all claims to it.

For practitioners new to competitive tabular machine learning, the resulting typology and prevalence map replace recipe-based folk knowledge with documented frequencies and a tested — and

largely cautionary — set of constraint–strategy couplings; for meta-research on machine learning practice, the study demonstrates a replicable multi-pass coding methodology applicable to other platforms or modalities.

1.5 Scope

The study covers tabular Kaggle competitions concluded between February 2022 and April 2026. The 45 winner entries include Playground Series Seasons 3 through 6, Tabular Playground Series competitions from 2022, and one Featured competition (ICR — Identifying Age-Related Conditions, August 2023) — so the corpus is predominantly synthetic-data Playground Series (44 of 45 entries), with a single real-data Featured competition. The primary unit of analysis is one solution per competition: the highest-placing top-three finisher with enough public documentation to code the solution (a writeup, and a notebook where available). In 30 of 45 competitions this was the first-place finisher; in the remainder the higher-placing solution lacked sufficient public documentation and the next-highest documented finisher was used (each entry's `finish_rank` is recorded). The study additionally collected 11 **near-winning controls** — rank 4–25 finishers from 11 of the same competitions who published a codeable solution (collection detailed in Section 3.2) — used to test whether feature engineering distinguishes winners from strong non-winners (Section 4.5) and to assess author overlap between the two tiers (Section 4.6). Excluded from scope are: time series and forecasting competitions, image and audio competitions, natural language processing competitions, multi-table join competitions where the winning solution did not first collapse to a single flat file, and the November 2022 TPS meta-blending competition (which involved no feature data).

1.6 Report Outline

Section 2 reviews related work in tabular machine learning benchmarking, competition-as-method research, and knowledge propagation in open communities, positioning the gap this study addresses. Section 3 documents the research design, data collection (winners and controls), the multi-pass coding methodology including the feature-engineering taxonomy, the audit and corrections workflow, analytical methods, the coding-reliability check, tools, and limitations. Section 4 reports the findings (§§4.1–4.9), beginning with an overview of the central result. Section 5 discusses and interprets them; Section 6 lists references.

---

2. Background and Related Work

2.1 Tabular Machine Learning Benchmarking

The dominant strand of academic work on tabular machine learning has focused on benchmarking model families against one another on fixed datasets. Caruana and Niculescu-Mizil established gradient-boosted trees as the strongest baseline on a range of tabular benchmarks more than a decade ago, a finding repeatedly reaffirmed by subsequent work. The AutoML benchmark family (Gijsbers et al.; Erickson et al. with AutoGluon) extended this comparison to automated pipeline frameworks. More recent work has revisited the question of whether neural network architectures specifically designed for tabular data can close the historical gap to gradient-boosted trees (Gorishniy et al.; Borisov et al.). These studies establish what "good" tabular models are technically, but they evaluate algorithms on standardized datasets in isolation, not in the competitive pipeline context where ensembles, custom feature engineering, and data exploitation strategies dominate. The present corpus offers a complementary view from competition practice: it lets us ask not

which model benchmarks best but which models actually win, and what — beyond model choice — separates the winning pipelines that surround them (Sections 4.2, 4.9).

2.2 Competition-as-Method Studies

A smaller body of work treats machine learning competitions themselves as objects of study rather than as benchmarks. The closest methodological precedent is the M5 forecasting retrospectives (Makridakis, Spiliotis, and Assimakopoulos), which characterized the methods used by winners across a large forecasting competition and distilled cross-cutting lessons; the Netflix Prize similarly produced detailed post-mortems of its single winning ensemble. These analyses, however, are largely narrative and confined to one competition or competition family, and none applies a structured, replicable coding scheme across a contemporary corpus. For tabular Kaggle specifically, knowledge of "what wins" remains at the level of individual blog posts and talks (Section 1.2). This study adapts that characterize-the-winners program to tabular competitions, and adds two elements absent from prior retrospectives: a 53-technique coding taxonomy and a near-winner control group.

2.3 Knowledge Propagation in Open Communities

How knowledge and credit flow through informal communities of practice has been studied in open-source software (von Krogh and von Hippel) and in the sociology of science. Most directly relevant is Merton's **Matthew Effect**: recognition tends to accrue to already-prominent contributors rather than to the originators of an idea, and success compounds (cumulative advantage). This predicts two patterns in a competitive community — a gap between the *originators* of techniques and the *canonizers* credited with them, and a small set of recurring high performers. To the author's knowledge, this lens has not previously been applied to competitive machine learning. Section 4.6 operationalizes it directly — through per-solution origination scores, citation counts, and win records — to examine both the originator-versus-canonizer gap and the recurrence of a small author pool in tabular Kaggle.

2.4 Stated Gap

Across these literatures, two questions stay unasked. Algorithm benchmarking (Section 2.1) shows what models are good, not what *wins*; competition retrospectives (Section 2.2) characterize winners but only within single competitions, narratively rather than through a structured comparative method; and the propagation lens (Section 2.3) is new to this setting. No prior work asks whether what winners do is *distinctive of winning* — which requires comparing them to strong non-winners — or treats the winning population as a *community* with measurable origination and recurrence. This study addresses both: it characterizes and classifies winning tabular solutions, tests whether their feature engineering distinguishes them from near-winning controls, and maps the community structure behind their techniques — and it does so with the structured, reliability-checked coding scheme that prior retrospectives lacked.

---

3. Methodology

3.1 Research Design

This study employed a systematic content-analysis design — a descriptive meta-analysis in the sense of aggregating and coding prior solutions rather than pooling effect sizes in the statistical sense of the term. Rather than collecting new experimental data, the research systematically aggregated and analyzed publicly available solution writeups and notebooks from completed Kaggle tabular machine learning competitions. The research was descriptive and taxonomic rather than confirmatory: the primary outputs were a four-paradigm typology of winning strategies, a 53-technique feature-engineering taxonomy with a group-level prevalence analysis, a comparison of winners against near-winning controls, a coupling-evidence table testing six promoted constraint-to-strategy patterns, and a set of frequency distributions and conditional cross-tabulations characterizing the corpus. The standard machine-learning project framework of training models and reporting prediction metrics does not apply here — the unit of analysis is one already-completed competition solution, not a new model trained by the researcher. Each guideline subsection below adapts the standard framework to a meta-analytic context.

The unit of analysis was one solution per competition: the **highest-placing** of the top-three finishers whose solution carried enough public documentation to code (a writeup at minimum, and a notebook where available). Documentation completeness was scored on a three-point `writeup_detail` scale (1 = discussion post only, 2 = notebook without full explanation, 3 = GitHub repository with detailed documentation) and used as an *inclusion threshold applied rank-first*, not as a quality ranking among finishers. The finishing rank of the selected solution (1st, 2nd, or 3rd) was recorded as a covariate (`finish_rank`); in 30 of 45 competitions the first-place finisher met the threshold, and in the remaining 15 the next-highest sufficiently-documented finisher was used. Because the rule is rank-first — taking the highest-ranked top-three finisher that clears a *minimum* documentation threshold — it does not preferentially select the most elaborately documented of a competition's top-three finishers. The corpus-level effect — that competitions in which no top-three solution was documented well enough to code drop out entirely, and that the studied "winner" is therefore the highest *documented* finisher rather than always the first-place one — is a genuine selection bias, addressed in Section 3.8. The analysis was cross-sectional: each competition contributed one entry to the dataset regardless of the number of active seasons or competitors.

3.2 Data Collection

Source and scope. Competition solutions were drawn from the Kaggle platform (kaggle.com). The study covered Playground Series Seasons 3 through 6 (January 2023 – April 2026), Featured competitions concluded in February 2022 or later, and three Tabular Playground Series competitions from 2022 (February, May, and June). The resulting dataset comprised 45 entries across six cohorts: PS S3 (n = 17), PS S4 (n = 10), PS S5 (n = 10), PS S6 (n = 4), TPS (n = 3), and Featured (n = 1).

Inclusion and exclusion criteria. A competition was included if (1) the competition data type was tabular (image, audio, NLP, and pure graph competitions were excluded); (2) the task was not a time series or forecasting problem; (3) the competition was not a multi-table join task unless the winning solution collapsed all tables into a single flat file before modeling; (4) at least a partial writeup existed for one of the top-three finishers (`writeup_detail` $\geq$ `1`); and (5) the selected solution used a tree-based model (XGBoost, LightGBM, or CatBoost) as a primary or significant ensemble component, or a neural network as the primary model. Neural-network-winning

solutions were retained because model selection between tree-based and neural approaches is itself a substantive observation. One competition was excluded: the November 2022 TPS meta-blending competition, which involved no feature data and no model selection task (reducing the candidate pool from 46 to 45 entries).

Data format and size. The final dataset was stored as a single Excel workbook (`data/kaggle_meta_analysis.xl` with six worksheets. The primary `Competition Data` sheet contained 45 rows and 40 columns after the corrections workflow described in Section 3.4 — the base schema plus the audit-era additions (`top_3_margin`, `distribution_shift_type`, and the split of `original_data_usage` into `external_original_available` and `external_original_use_mode`). Column categories included competition identity fields (`competition_ref`, `finish_rank`, `writeup_detail`, `code_url`), dataset characteristics (`n_rows`, `n_features`, `pct_missing`, `max_cardinality`, `has_categorical`, `feature_type_dominant`, `target_type`, `metric`, `distribution_shift`, `distribution_shift_type`, `top_3_margin`), model fields (`dominant_base_model`, `primary_model`, `models_used`, `best_single_model`), preprocessing fields (`missing_data_strategy`, `encoding_strategy`, `scaling`, `outlier_treatment`), pipeline fields (`cv_strategy`, `hyperparameter_tuning`, `ensemble_method`, `original_data_usage`, `external_original_available`, `external_original_use_mode`), and feature engineering (`fe_techniques`). A second sheet, `Paradigm & Attribution`, added 18 columns of Pass 2 re-evaluation data, including paradigm classification, origination score, cited community members, and notable commenters. The remaining sheets (`Codebook`, `Example (icr)`, `Data Quality Audit`, `Corrections Log`) documented the schema, an annotated example entry, the identified data-quality issues, and the cell-level changes applied during the audit.

Access method. Competition metadata (competition name, slug, evaluation metric, leaderboard) was collected via the Kaggle REST API (Python package `kaggle` v1.6) using a registered Kaggle account. Discussion post titles and vote counts were retrieved via `api.competitions_list()` and `api.kernel_list()`; however, the Kaggle REST API does not expose the body text of discussion posts, and `kernels_list()` returned zero results for many competition slugs, because winning solutions are commonly shared via GitHub links or file attachments rather than as published Kaggle kernels. As a result, the actual collection of solution content required extensive manual review: writeups were read directly from competition discussion pages and writeup tabs on Kaggle, and notebooks were either pulled via `api.kernels_pull()` when the winner had published a public Kaggle kernel, or downloaded manually from GitHub links. Dataset-level statistics (`n_rows`, `n_features`, `pct_missing`, `max_cardinality`) were auto-populated by downloading each competition's `train.csv` file via the Kaggle API and computing summary statistics in Python.

Near-winning controls. To test whether feature engineering distinguishes winners from strong non-winners (Section 4.5), a second, smaller corpus of 11 *near-winning controls* was collected. A control is a rank 4–25 finisher who published **both** a writeup **and** a code notebook — a deliberately stricter documentation bar than the winners corpus, which required only a writeup at minimum. Selection proceeded competition by competition: rank-4–25 finishers were checked against this both-documents requirement, and qualifying finishers were included. The requirement was met rarely enough that it yielded exactly one control per competition, for 11 controls across 11 distinct competitions — each therefore pairable to that competition's own winner in Section 4.5. The rank-4–25 band was chosen as the closest documented non-winning tier: rank 1–3 entries are effectively co-winners, frequently within a 0.00001 leaderboard margin, while documented solutions below rank ~25 become scarce. Controls were coded only on the Pass 3 feature-engineering taxonomy (Section 3.4), not on the full Pass 1/Pass 2 schema, because the comparison of interest is feature-engineering intensity and composition. This is a purposive, non-random sample, and the both-documents

requirement is itself the mechanism behind a key property of the pool: it is rare enough that the controls concentrate among the documented elite and overlap the winners corpus in authorship — a property treated as a finding (Section 4.6) and a limitation (Section 3.8) rather than a nuisance.

Licensing. All competition writeups, notebooks, and solution discussions are public submissions on Kaggle, freely viewable without authentication. Competition datasets are provided under each competition's own data terms; the study used only metadata computed from those datasets (row counts, column counts, missing value rates) rather than the raw data itself.

3.3 Data Preprocessing

Preprocessing in this study referred to the cleaning and normalization steps applied to `kaggle_meta_analysis.xlsx` — the meta-dataset of collected competition solutions — rather than to any machine learning input data.

Value normalization. All multi-value categorical fields (for example, `fe_techniques`, `models_used`, `ensemble_method`) used a semicolon character (`;`) as a canonical separator. Values were normalized to lowercase with underscores (for example, `TargetEncoder` → `target_encoding`; `One Hot` → `one_hot_encoding`). Common aliases encountered across writeups were mapped to their canonical equivalents using a manually maintained alias table defined in the `Codebook` sheet.

Sentinel coding. Fields that a solution did not describe at all were coded as `not_described`. Fields that were genuinely inapplicable for a given solution (for example, `scaling` for a gradient-boosted tree model that explicitly stated no scaling was used) were coded as `not_applicable`. These two values were distinguished to avoid conflating missing data with a meaningful absence.

Derived fields. Three fields were derived rather than directly coded. `dominant_base_model` was derived from `primary_model` and `models_used`: if the primary model was one of XGBoost, LightGBM, or CatBoost, or if GBM variants comprised the majority of models listed in `models_used`, the entry was assigned `dominant_base_model = gbm`; entries with a neural network as primary or dominant were assigned `neural_network`; entries with linear models as primary were assigned `linear`. `top_3_margin` was computed automatically from the `leaderboard.json` file for each competition as the absolute difference between the rank-1 and rank-3 private leaderboard scores. `distribution_shift_type` was assigned to all 45 entries by per-writeup re-evaluation (Pass 2, Section 3.4): entries with `distribution_shift = TRUE` received the specific shift category (`temporal` or `covariate`), entries with `distribution_shift = FALSE` received `none`, and entries where shift was not described received `not-applicable`.

Train/test split. No train/test split was applied to the meta-dataset itself. The corpus is treated as the population of recent winning tabular Kaggle solutions rather than as a sample for prediction modeling.

3.4 Analytical Methods

The analytical strategy used a three-pass coding design — field-level coding (Pass 1), qualitative re-evaluation (Pass 2), and a feature-engineering taxonomy (Pass 3) — with coding reliability assessed separately (Section 3.6).

Coding procedure. Primary coding across all three passes was AI-assisted: a large language model produced the initial codes for each entry directly from its writeup and notebook, under

researcher direction. The researcher authored the schema, codebook, alias table, and feature-engineering taxonomy, directed the coding, and adjudicated flagged or ambiguous cases. Because AI-assisted coding can introduce systematic error, the reliability of this coding was not assumed but assessed through a blind re-code on a subsample (Section 3.6).

Pass 1: Structured field-level coding. The first pass applied the 38-variable schema described in Section 3.2 to each of the 45 entries. Each solution was coded directly from its writeup and (where available) its published notebook, with values normalized to the canonical Codebook entries. Pass 1 produced the `Competition Data` sheet. Frequency analysis on Pass 1 fields was performed using `pandas.DataFrame.value_counts()` and `DataFrame.describe()`; cross-tabulations of pairs of fields used `pandas.crosstab()`. This pass captured *what techniques were used* in each solution.

Pass 2: Per-writeup qualitative re-evaluation. Pass 1 alone could not distinguish whether a technique listed in `fe_techniques` was originated by the winner or inherited from a publicly shared notebook by a community member. To capture origination, attribution, and the qualitative *shape* of each winning strategy, a second pass was conducted: each of the 45 writeups was re-read using a nine-section template covering identifiers, dataset characteristics, what Pass 1 records, public-notebook reuse, what the author actually originated, dataset constraints that shaped the strategy, code-versus-writeup verification, headline finding, and surprising or unusual observations. Pass 2 produced (1) one per-writeup document per entry, archived under `analysis/writeup-reevaluation/`; (2) an INDEX document synthesizing cross-cutting patterns; and (3) the 18-column `Paradigm & Attribution` worksheet, which assigned each entry to one of four winning paradigms — `ensemble-stacking`, `single-model-FE`, `lookup-exploit`, `problem-fit-NN` — or to `mixed` or `community-template-tweak`. Pass 2 also recorded an `origination_score` (0–3 ordinal: 0 = pure fork, 3 = mostly original), cited community members, academic papers cited, canonized techniques used, and a one-line `winner_unique_edge` summary capped at 200 characters.

Pass 3: Feature-engineering taxonomy. A third coding pass applied a 53-technique feature-engineering taxonomy to each of the 45 winners and the 11 controls. The taxonomy was developed iteratively from the corpus and organized into 11 families (Groups A–K): categorical encoding, numeric transformations, interactions and combinatorial features, aggregates and groupby features, domain/temporal features, external-original-derived features, learned/advanced features, text and string-pattern features, model-derived features, feature selection, and meta indicators. Each of the 53 techniques was coded as a boolean (present/absent) for each entry from its writeup and notebook, stored as one row per entry in `analysis/pass3-fe-taxonomy/`. A per-entry technique count (`n_fe`) was computed as the number of TRUE techniques *excluding* the two meta-indicator flags (explicit-minimal-feature-engineering and forked-uncatalogued), so that `n_fe` reflects the author's own catalogued techniques rather than provenance flags. The taxonomy yields a prevalence map of which technique families winners use (Section 4.4) and supports the winner-versus-control comparison (Section 4.5); group-level prevalence is computed as the percentage of entries using at least one technique within a family. All quantitative feature-engineering results are reported at the family (group) level rather than the individual-technique level, for reasons established by the reliability analysis (Section 3.6).

Data-quality audit and corrections workflow. During Pass 2, cell-level inconsistencies in Pass 1 were identified — for example, `n_rows` for one entry was coded as 300,000 when the writeup stated that 4 million training rows had been loaded (one of two CSV files had been counted in isolation). A dedicated `Data Quality Audit` worksheet catalogued these issues by severity, and corrections were applied in severity order with a full diff trail (old value, new value, reason, source) recorded in a

separate `Corrections Log` worksheet. The audit also motivated two schema changes: the addition of `top_3_margin` and `distribution_shift_type`, and the split of `original_data_usage` into `external_original_available` and `external_original_use_mode` to disambiguate availability from use mode.

Coupling-evidence framework. The INDEX from Pass 2 promoted six recurring constraint-to-strategy patterns (each observed in at least two cases) as candidate couplings. To test each coupling quantitatively, each was operationalized as a pair of filter functions over the merged Pass 1 + Pass 2 dataset: a *constraint* filter (which entries are testable for this coupling) and a *strategy* filter (which entries chose the predicted strategy). For each coupling, the analysis computed the number of entries supporting the coupling (constraint and strategy both true), the number contradicting it (constraint true, strategy not chosen), the number where the coupling was not applicable (constraint false), and a support rate (`n_supporting / n_testable`). A verdict tier was assigned: `strong` (support rate ≥ 0.75), `weak` (0.50–0.74), `contradicted` (< 0.50), or `undersupported` (`n_testable < 3`).

3.5 Evaluation Metrics

Because the unit of analysis is one already-completed competition solution and no new model is trained, the standard machine learning evaluation metrics (RMSE, accuracy, F1, etc.) do not apply to the meta-analysis itself. The evaluation framework instead consisted of four indicators.

Feature-engineering prevalence and winner–control comparison. For the feature-engineering analysis (Sections 4.4–4.5), the primary indicators were group-level prevalence — the percentage of entries using at least one technique within a family — and, for the winner-versus-control comparison, the within-competition paired difference in technique count (`n_fe`) across the 11 winner–control pairs. The paired difference was evaluated with a sign test on the per-pair direction (which member did more feature engineering); group-level prevalence was compared descriptively between the winner and control corpora.

Field completeness rate. For each schema field, the proportion of entries with a substantive value (anything other than `not_described`, `not_applicable`, or null) was reported. Fields with fill rates below 60% were flagged as too sparse to support reliable conditional analysis.

Paradigm distribution and stability. The distribution of paradigm assignments across the corpus (and by era cohort) was reported, with attention to (1) which paradigms recur across multiple eras and (2) which paradigms cluster in specific eras only. Paradigms appearing in at least two eras were treated as candidates for generalization; paradigms confined to a single era were treated as descriptive observations only.

Coupling support rate. Defined as the number of entries supporting a promoted coupling divided by the number of entries where the coupling is testable. Coupling support rates were the indicator used for the coupling analysis specifically — whether each of the six promoted couplings is empirically supported by the corpus. The `strong` threshold of 0.75 was set a priori based on the small sample size (typical `n_testable` between 5 and 40), with the intention that couplings reaching this threshold could be defended as documented patterns rather than informal observations.

A significance level of $\alpha = 0.05$ was used where chi-square tests of independence were applicable on 2×2 contingency tables, and a sign test was applied to the paired winner–control comparison (Section 4.5). Given the exploratory nature of the analysis and the small sample sizes (N = 45

winners; 11 paired comparisons), chi-square, Fisher's exact-test, and sign-test p-values are reported as supporting evidence rather than as definitive tests of hypotheses.

3.6 Coding Reliability

The primary codings were produced with AI assistance under researcher direction (Section 3.4). The reliability assessment focused on **Pass 3**, the feature-engineering taxonomy, because its 53 fine-grained boolean judgments carry the most coder discretion and are the most subjective coding in the study. Pass 1's schema fields are largely factual extraction (row counts, metrics, models), where coder discretion is minimal and accuracy — not inter-coder agreement — is the relevant concern; these were validated through the data-quality audit (Section 3.4) rather than a reliability re-code. Pass 2's interpretive codings (paradigm, `origination_score`) cover winners only and were single-coded, a limitation acknowledged in Section 3.8. A reliability check was therefore performed on a stratified Pass-3 subsample to quantify how reproducibly the taxonomy can be applied.

Design. The researcher conducted a blind *intra-rater* re-code — re-coding the sample by hand, without reference to the AI-assisted primary codings — on a stratified 12-entry sample ($\approx 21\%$ of the combined 56-entry winners-plus-controls corpus), drawn to span eras (one each from the 2022 Tabular Playground series and Seasons 3–6) and source-confidence levels, and mixing eight winners with four controls. The re-code worked from the original notebooks and writeups plus the schema definitions only — blind to the primary codings, the coding notes, and the data sheets. Agreement was then computed between the blind re-code and the primary codings. Because the same researcher both directed the primary coding and performed the re-code, this measures the reproducibility of the scheme in one coder's hands (intra-rater), not agreement between fully independent coders (inter-rater), and the figures below should be read accordingly.

Results. Raw per-cell agreement was high (94.5% at the 53-column level), but this figure is inflated by the large number of jointly-absent techniques and is reported only for completeness. Chance-corrected agreement gives the interpretable picture, and it depends strongly on resolution:

Resolution	Cohen's κ	Gwet's AC1	Positive agreement
53-column	0.60 (moderate)	0.94	46%
11-group	**0.65 (substantial)**	**0.78**	58%

At the full 53-column resolution Cohen's κ is only moderate (0.60); Gwet's AC1 (0.94) is reported but is known to over-correct under extreme prevalence skew and is not relied upon. Collapsing the 53 columns into the 11 technique families (Section 3.4) raises κ to 0.65 — *substantial* on the Landis–Koch scale — and brings κ and AC1 into agreement (0.65 and 0.78), indicating a non-paradoxical estimate. Because reliability is substantial only at the family level, **all quantitative feature-engineering results are reported at the group level** (Sections 4.4–4.5); the 53-column resolution is used for qualitative description only. These figures are computed against the *pre-adjudication* coding and are reproduced from the committed data by `analysis/pass3-fe-taxonomy/stats.py`; folding the adjudicated corrections (below) back in would raise agreement circularly and is deliberately avoided.

Adjudication. Disagreements were adjudicated against source materials. They were bidirectional: the primary codings had missed techniques the blind re-code caught (e.g., a `carat`^{3} polynomial and a per-row max/min feature in one entry; an original-target-mean feature in another), while

the blind re-code had missed techniques the primary codings caught (e.g., frequency encoding and pairwise interactions a "no-feature-engineering" reading overlooked). Four winner entries received source-verified corrections (most substantially an OpenFE / Sequential-Feature-Selection pipeline that had been mis-coded), applied to the dataset with provenance notes; a single coding rule was fixed for the forked-uncatalogued indicator (it applies to forked *feature pipelines*, not to ensembling another model's out-of-fold predictions). The positive-agreement column above (46% at column level, 58% at group level) is reported as an honest reminder that even at the family level the primary coding and the blind re-code converged on which families were present only ~58% of the time; fine-grained feature-engineering coding is sensitive to coder thoroughness, a limitation the group-level reporting is designed to absorb.

3.7 Tools and Technologies

All work was performed in Python 3.13, with dependencies managed by Poetry (`pyproject.toml` at the project root). Because the study analyzes completed solutions rather than training new models, the toolchain centers on data wrangling, the Kaggle API, spreadsheet I/O, and visualization rather than modeling frameworks. The tools actually used were:

Tool	Version	Purpose
Python	3.13	Primary language for all collection, coding, and analysis scripts
Poetry	2.x	Dependency and environment management (`pyproject.toml`)
pandas	2.2	Pass-1 frequency tables (`value_counts`) and cross-tabulations (`crosstab`)
numpy	2.x	Numerical operations and figure data preparation
matplotlib	3.9	Report figures (Figures 3–6)
seaborn	0.13	Exploratory visualization in the EDA notebook (not the report figures)
openpyxl	3.1	Reading and writing the `kaggle_meta_analysis.xlsx` meta-dataset
kaggle (API)	1.6	Competition metadata, leaderboard retrieval, and notebook pulls
requests + BeautifulSoup	2.32 / 4.12	Scraping writeup and discussion content the Kaggle API does not expose
anthropic (Python SDK) + Claude Code CLI	Claude Opus 4.7–4.8	AI-assisted coding of the corpus (Pass 1–3) under researcher direction (Section 3.4), plus development support

Tool	Version	Purpose
Jupyter	1.1	EDA and reanalysis notebooks (`notebooks/archive/01_eda.ipynb`, `notebooks/02_reanalysis.ipynb`)
Git / GitHub	—	Version control
Python standard library	3.13	`csv` / `json` I/O, `statistics` (medians), `collections.Counter` (frequency counts)

Reliability statistics (Cohen's κ, Gwet's AC1, positive agreement) and the paired sign test are reproduced from the committed data by `analysis/pass3-fe-taxonomy/stats.py`; the coupling chi-square and Fisher's exact tests were computed with custom routines in `notebooks/02_reanalysis.ipynb`. These computations use only the Python standard library — the project does not depend on a dedicated statistics package such as SciPy. The `pyproject.toml` also declares modeling libraries (scikit-learn, XGBoost, LightGBM, CatBoost) retained from the initial project scaffold; these are not used by the meta-analysis, which trains no models. The project was developed on Windows 11 Pro using PowerShell as the primary shell.

3.8 Limitations

Small sample size. With N = 45 entries, the dataset is underpowered for sub-group analyses. The ensemble-stacking stratum (n = 28) supported reasonably reliable frequency and cross-tabulation analysis; the smaller paradigm strata (single-model-FE n = 6, lookup-exploit n = 4, problem-fit-NN n = 3, community-template-tweak n = 3, mixed n = 1) supported descriptive characterization only. Results within smaller strata are reported with explicit small-n caveats.

Single-researcher coding, with AI assistance. Primary coding across all three passes was AI-assisted under researcher direction (Section 3.4); no second independent coder was involved, so the reliability check is *intra-rater* rather than inter-rater. For the Pass 3 feature-engineering taxonomy, the researcher performed a blind re-code of a stratified subsample (Section 3.6), yielding substantial family-level agreement ($\kappa = 0.65$) and only moderate 53-column agreement ($\kappa = 0.60$) — which is why feature-engineering results are reported at the group level. Pass 1 and Pass 2 were not separately re-coded. Pass 1 is largely factual extraction, and its values were checked during the Pass 2 re-evaluation through the data-quality audit (Section 3.4), which surfaced and corrected cell-level inconsistencies. Pass 2's interpretive codings (paradigm and `origination_score`) were single-coded and cover winners only; the genuinely subjective element left unmeasured is `origination_score`, an ordinal judgment — but the recurrence finding in Section 4.6 rests on *factual* fields (author handles, win records, citation counts) rather than on that score, so it is unaffected.

Writeup availability bias. The study required at least a partial writeup for inclusion. Competitions where no top-three finisher published any solution documentation were excluded from the candidate pool before screening. If solutions with published writeups differ systematically from those without — for example, if winners who publish tend to use more standard or reproducible techniques — the resulting dataset may not be fully representative of all winning approaches on the platform.

Limited monetized competition coverage. The original study design planned to compare

monetized (Featured) competitions against playground (non-monetized) competitions as a covariate. Only one monetized competition entry was collected (ICR — Identifying Age-Related Conditions), making this comparison infeasible. All distributional analyses are conducted on the combined corpus.

Detection-limited couplings. The strategy filters used in the coupling-evidence framework (Section 3.4) operate on the 200-character `winner_unique_edge` field of the Pass 2 sheet, which captures the single most distinguishing element of each solution. For couplings whose strategy signal is subtle (for example, "validation discipline" or "deliberate no-FE decision"), the strategy may be present in the underlying writeup but absent from the 200-character summary, producing a *contradicted* verdict that is actually a *detection-limited* one. This limitation is noted explicitly when interpreting contradicted couplings in Section 4.

Documentation self-selection in the control sample. The winner–control comparison (Section 4.5) addresses, but does not fully close, the absence of a non-winner baseline. The 11 controls are filtered twice: by outcome (a rank 4–25 finish, the top ~1% of a competition) and by documentation (a published writeup). The second filter is not independent of competitor identity — among lower-ranked entries, writeups are produced almost exclusively by habitual, brand-conscious competitors — so the control pool over-samples a small recurring elite and in fact shares an author with the winners corpus in 5 of 11 cases (Section 4.6). Consequently, the comparison characterizes feature-engineering differences between winners and *documented near-winners*, and is silent on the difference between winners and *typical* competitors, who place mid-field and rarely publish and are therefore invisible to a writeup-based corpus by construction. Sampling deeper into the leaderboard would not relax this filter (it tightens at lower ranks); escaping it would require a different instrument (e.g., public notebooks sampled by leaderboard quartile, or solution descriptions from non-writeup-authors), which is noted as future work. The winners corpus is subject to the identical documentation filter, so the winner–control comparison is internally consistent; the limitation is that neither sample represents a typical competitor.

Synthetic-data corpus. Forty-four of the 45 entries are Kaggle Playground Series competitions, whose datasets are synthetically generated from real-world sources. Several headline patterns — the dominance of target encoding, the lookup/exploit paradigm, and the use of the original source dataset as external features — are at least partly artifacts of synthetic generation and may not transfer to real-world tabular problems or to Featured competitions (n = 1 here). Generalization beyond the synthetic-data Playground setting is therefore unestablished, and is discussed further in the Discussion (Section 5).

Temporal scope and generalizability. The dataset covers February 2022 through April 2026. Kaggle Playground Series Season 6 entries (n = 4) showed a qualitative shift toward larger ensembles with more than 100 base models and explicit use of large-language-model agents in the coding workflow (most prominently in one entry's "KGMON Playbook" methodology). The four-paradigm typology derived primarily from earlier-season entries may require extension or modification to accommodate continued evolution of this emerging competitive paradigm.

Attribution-conflation resolved in Pass 2 but not in Pass 1. The Pass 1 `fe_techniques` field captures *which techniques were used* in each solution but cannot distinguish techniques originated by the winner from techniques inherited from a publicly shared community notebook. This systemic limitation of Pass 1 is addressed by the Pass 2 `origination_score` and `cited_community_members` fields, but it means that any analysis drawn directly from Pass 1's `fe_techniques` column reports technique *adoption*, not technique *origination*. The two questions are kept separate throughout

Section 4.

---

4. Results

4.1 Overview: The Central Surprise

Feature engineering is widely held to be the decisive craft in tabular machine learning, so one might expect the winners of tabular competitions to be distinguished by especially extensive or sophisticated feature work. The corpus does not bear this out.

The results below build the case in stages. Sections 4.2–4.3 establish what winning solutions look like structurally: a typology dominated by ensemble stacking (4.2), and a cautious test of the constraint-to-strategy couplings promoted from the qualitative pass, most of which the corpus does not strongly support (4.3). Sections 4.4–4.5 turn to feature engineering: winners' feature work is a small shared core rather than an elaborate practice (4.4), and feature-engineering *volume* does not cleanly separate winners from strong non-winners — the comparison is sensitive to framing, with only learned features showing a consistent winner-favoring signal (4.5). Section 4.6 characterizes the documented top as a small recurring guild, distinguished more by who competes than by technique inventory; Section 4.7 traces a recent shift toward very large ensembles and LLM-assisted workflows; and Sections 4.8–4.9 give the descriptive composition of the corpus. The recurring theme is that *winning is less about feature-engineering volume than the folklore suggests, and more about paradigm fit, a small set of high-leverage techniques, and membership in a small recurring guild.*

4.2 Paradigm Typology

The four winning paradigms identified by Pass 2 re-evaluation are defined as follows:

- **Ensemble-stacking** (n = 28, 62%): The solution combines multiple base models (typically 5–100+) using one or more stacker layers, where the stacker is a learned model (Ridge regression, hill-climbing weight optimization, mean blending, or a learned meta-learner such as AutoGluon or CatBoost). The base models are themselves often diverse — gradient-boosted trees, neural networks, classical kernel methods, and sometimes AutoML frameworks — and the winning edge typically comes from the diversity-and-stacking combination rather than from any single base model.
- **Single-model heavy-FE** (n = 6, 13%): The solution uses a single model class (typically XGBoost, sometimes LightGBM or a kernel method) with extensive feature engineering, often producing hundreds of engineered features from a small base feature set. No ensembling is used. This paradigm is most viable when training data is large enough to absorb the engineered features without overfitting, or when the dataset is small and ensembling would add noise.
- **Lookup-exploit** (n = 4, 9%): The solution exploits a property of the synthetic-data generation process — typically that the generator preserves identifiable (feature, target) pairs from a public original dataset, or that two training rows with identical features have opposite labels — to predict via direct key-matching rather than via a learned model. Each of the four cases exploits the generator differently: identity-lookup, inversion-lookup, distance-via-generator-flaw, and exact-row-join via an exact-solution feature.
- **Problem-fit neural network** (n = 3, 7%): The solution uses a custom neural network architecture matched to a specific structural property of the data — a two-branch architecture

mirroring the disconnected components of an interaction graph; a denoising autoencoder used because the prediction task itself is missing-value imputation; or a Variable Selection Network applied at extreme small-data scale with heavy regularization. All three appear in 2022–2023; no problem-fit-NN paradigm wins appear in the corpus from 2024 or later.

The four paradigms above are the corpus's primary strategic archetypes. Two residual categories hold the four remaining solutions that do not fit a primary archetype cleanly — one fork-dominated, one genuinely mixed:

- **Community-template-tweak** (n = 3, 7%): The solution is built primarily on top of a forked public notebook contributed by another community member, with one meaningful addition by the winner (typically domain-specific feature engineering or a small meta-architecture change such as a two-binary-classifier reframing of a three-class problem).
- **Mixed** (n = 1, 2%): The solution does not fit cleanly into a single paradigm; in the one identified case, the winner uses a small mean-blended ensemble at small data scale while explicitly avoiding feature engineering, combining elements of both the single-model heavy-FE and ensemble-stacking categories.

An important descriptive finding from Pass 2 is that no entry in the corpus has an `origination_score` of 0 (pure fork). Even the most fork-heavy paradigm (community-template-tweak) contains at least one meaningful original contribution per winner — domain-driven feature engineering, a meta-architecture choice, or a deliberate post-processing step. The community-template-tweak paradigm is therefore "fork-plus-meaningful-tweak," not "fork verbatim."

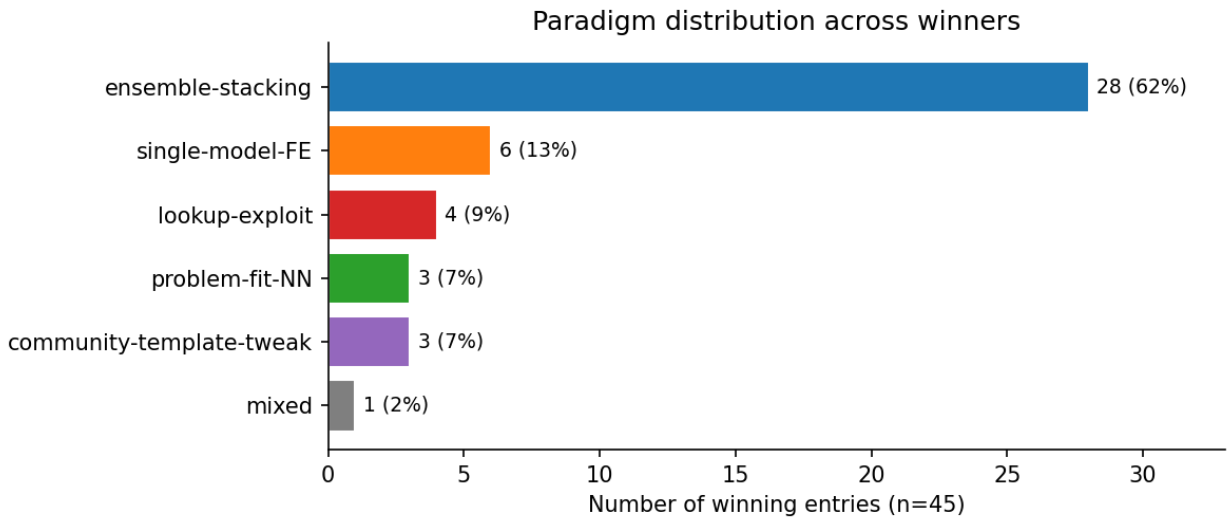


Figure 1. Paradigm distribution across winners.

4.3 Coupling Evidence

Beyond *which* paradigm a winner adopts (Section 4.2), the qualitative re-evaluation surfaced recurring practitioner claims that *specific dataset constraints* drive *specific strategy choices* — folklore couplings of the form "when X holds, winners do Y." Testing which of these survive is part of characterizing what reliably distinguishes winning approaches, and — as with the feature-engineering folklore examined later — several widely repeated couplings turn out not to hold. Six such couplings identified during Pass 2 were tested quantitatively; Table 1 summarizes the results.

Table 1. Coupling-evidence verdicts.

Coupling	n_supporting	n_contradicting	n_testable	Support rate	Verdict
C3: Linear stacker dominance	32	8	40	80%	Strong
C6: Distribution shift → custom CV	4	1	5	80%	Directional (n = 5)
C2: Heavy citations + recurring author → fork-heavy win	3	5	8	38%	Contradicted
C4: Available external original → use as columns	11	19	30	37%	Contradicted
C5: Small data (< 50K rows) → no-FE single-model	3	10	13	23%	Contradicted
C1: Photo-finish margin → validation discipline	6	21	27	22%	Inconclusive (detection-limited)

Only one coupling clears the strong-evidence threshold on a substantial sample, and it largely confirms established practice. **Linear-stacker dominance** is supported by 32 of 40 entries that used an ensemble stacker: when winners ensemble, they predominantly choose a linear stacker (Ridge, mean blend, weighted blend, hill climbing, or LAD regression) rather than a nonlinear one. This validates a well-known ensembling heuristic — simple stackers generalize better than complex meta-learners — rather than revealing a novel coupling; the eight contradicting cases include three entries where the winner explicitly chose a nonlinear stacker (CatBoost, AutoGluon, or a neural network) after a head-to-head comparison. **Distribution-shift-driven custom CV** is directional only: four of the five entries with `distribution_shift_type` ∈ {temporal, covariate} used a custom CV scheme, but on five testable entries the pattern is suggestive at best, not robust.

Three couplings are clearly contradicted by the data, and a fourth (C1) is inconclusive for measurement reasons. The most striking contradiction is C4 ("use available external original as columns"): of the 30 entries where an external original dataset is available, only 11 use it as columns, features-derived, both, or lookup; the remaining 19 use it only as additional training rows (rows-only concate-

nation). The widely cited "use as columns" pattern is idiosyncratic to one prominent author and does not generalize to the community. C5 ("small data → no-FE single model") is contradicted because seven of ten small-data wins still ensemble multiple models, even though no-FE-single-model is the explicit philosophy of one of the most cited Kaggle authors. C1 ("photo-finish margin → validation discipline") is *not* counted as contradicted: its low support rate reflects detection limits rather than absence, because the strategy detector keys off the 200-character `winner_unique_edge` field, which often does not surface validation-discipline as the distinguishing element even when the underlying writeup describes it. C2 ("heavy citations + recurring author → fork-heavy win") is contradicted on a small sample: even among the eight entries that are both author-recurring and heavily citing, only three have origination scores of 2 or below, meaning recurring authors who cite many sources still produce mostly-original work.

On balance, the constraint-to-strategy folklore is weakly supported at best: four of the six couplings are contradicted, and the two that are not are either a restatement of standard ensembling practice (C3) or too thinly sampled to rely on (C6, n = 5). The couplings are therefore reported as a cautionary characterization of practitioner folklore, not as a set of validated decision rules.

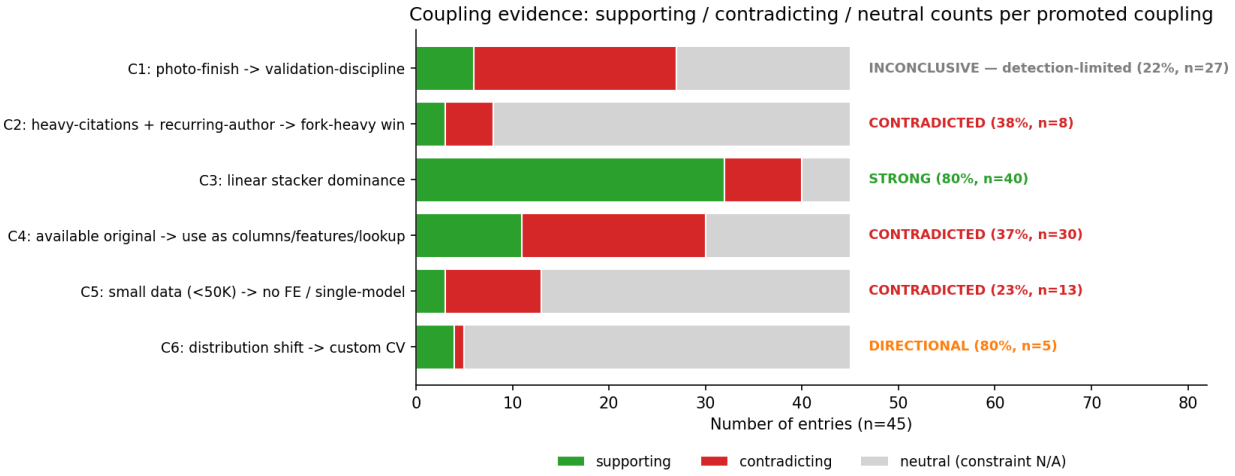


Figure 2. Coupling evidence: supporting / contradicting / neutral counts per promoted coupling.

4.4 Winning Feature Engineering Is a Small Shared Core

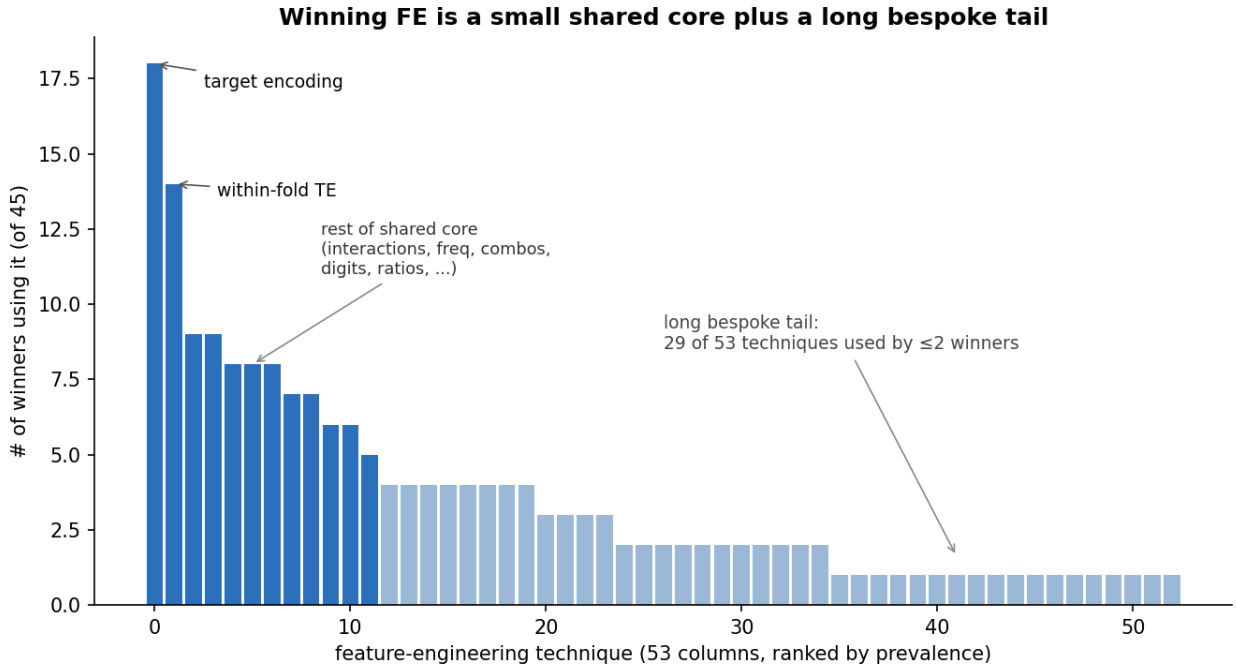
The feature-engineering taxonomy (Section 3.4) reveals that winning solutions concentrate on a small shared set of techniques rather than an elaborate or idiosyncratic craft. **Target encoding is the single most common technique**, used by 18 of 45 winners (40%); of those, 14 (31% of all winners) use its leak-safe within-fold variant. The next tier — multiplicative and additive interactions, higher-order categorical combinations, frequency encoding, and digit features — is used by roughly 9–13% of winners each. Beyond this core, technique usage falls off sharply into a long tail: **29 of the 53 catalogued techniques are each used by two or fewer winners** (Figure 3). The tail's length partly reflects the taxonomy's resolution — a 53-technique catalog built to capture the corpus's full diversity registers many rare techniques by construction — but the sharp concentration at the head (target encoding at 40%, the next tier in single digits) is a genuine feature of the distribution, not an artifact of how finely techniques are split. Winning feature engineering is thus better described as a small shared core plus a long bespoke tail than as a uniformly sophisticated practice.

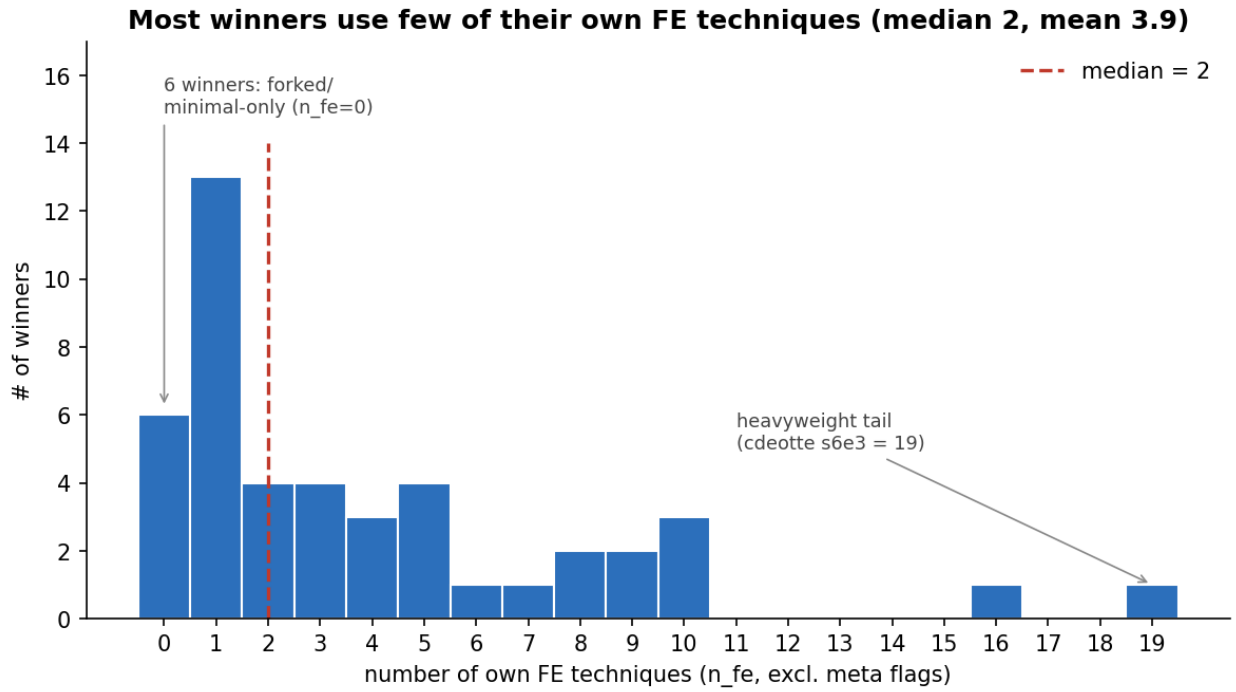
The *amount* of feature engineering per winner is modest, but it must be read against documentation depth: because `n_fe` counts only techniques observable in the writeup and notebook, it is a lower bound that scales with how completely each solution was documented. The effect is large (Table 2). Among the 22 winners whose feature engineering can be verified against a published notebook, the median uses **4** of their own techniques (mean 5.0); among the 23 documented by writeup or notes alone — where much feature engineering is simply unobservable — the median is **1**. The corpus-wide median of 2 therefore understates true intensity and should be read as a floor.

Source available	n	median `n_fe`	mean	max
Notebook + writeup	22	**4**	5.0	16
Writeup or notes only	23	1	2.9	19
All entries	45	2	3.9	19

Table 2. Own-technique count (`n_fe`) by documentation depth.

Even at the better-documented figure the count is modest and the distribution right-skewed: among notebook-backed winners most use one to five own techniques, with a small heavyweight tail (the corpus maximum is 19) (Figure 4). Six winners use no own techniques at all, relying entirely on forked public features or an explicit no-feature-engineering strategy. Crucially, the small-core-plus-long-tail structure holds *within* the well-documented subset — so it is a property of winning practice, not merely an artifact of sparse documentation, though terse writeups do exaggerate how sparse the practice appears. Elaborate feature engineering is, on this evidence, neither typical of nor necessary for a winning tabular solution.





4.5 Feature Engineering and the Winner–Control Gap

Whether feature engineering separates winners from strong non-winners depends critically on how the comparison is framed, and the two natural framings disagree — a discrepancy that is itself informative.

The aggregate comparison is confounded. Compared in aggregate — all 45 winners against all 11 near-winning controls at the technique-family level (Figure 5) — the controls match or exceed the winners on nearly every family: categorical encoding (64% vs 44%), numeric transforms (55% vs 38%), external-original features (36% vs 18%), and others, with the median control using 3 own techniques to the median winner's 2. Taken at face value this would suggest that feature-engineering volume does not distinguish winners. But the aggregate compares two differently-composed competition sets: the 11 competitions that happen to have codeable controls are disproportionately feature-engineering-intensive — their winners have a median `n_fe` of 6 (up to 19), well above the corpus-wide winner median of 2 (Section 4.4) — while the full 45-winner pool is pulled down by minimal-FE winners from *other* competitions. The aggregate therefore measures a competition-mix difference, not a winner-versus-near-winner difference.

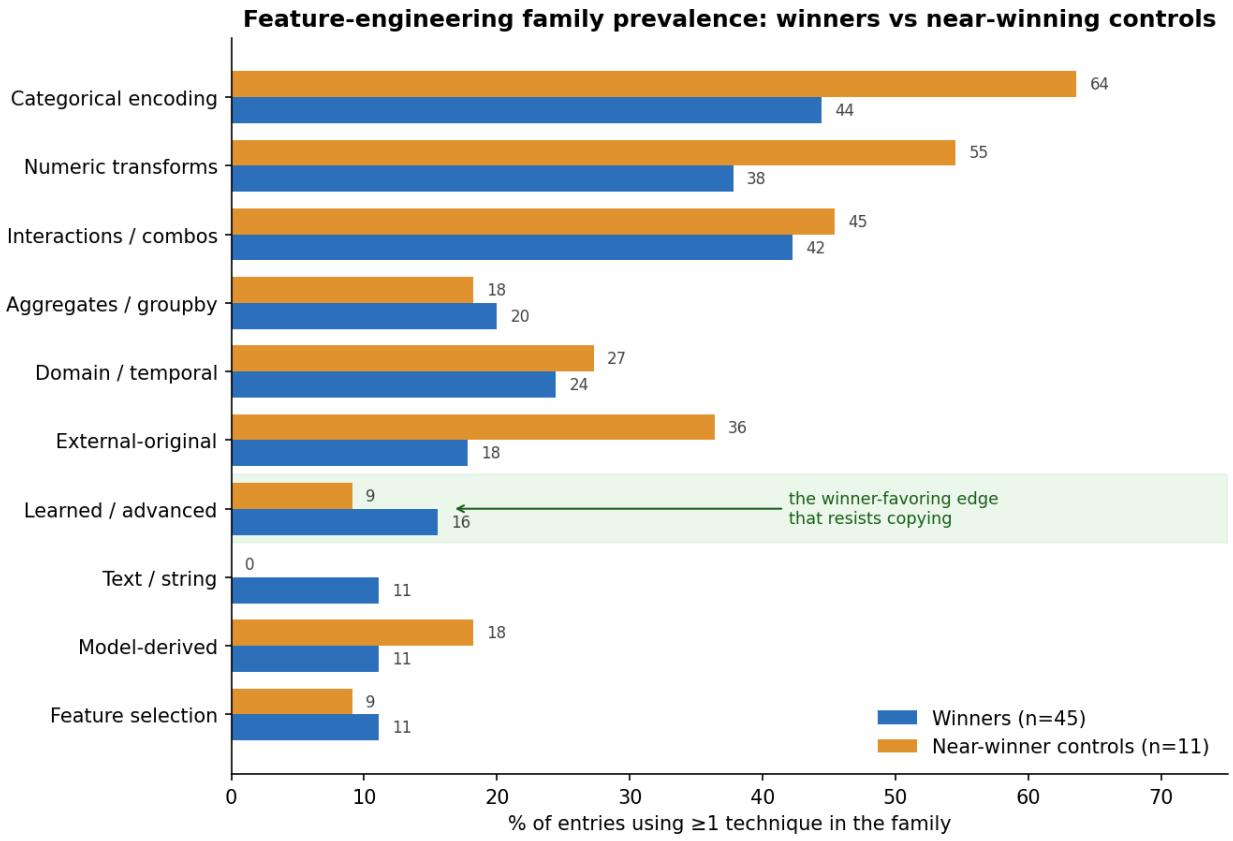
The paired comparison reverses it. A within-competition paired comparison — each control against the winner of its *own* competition (Figure 6) — inverts the aggregate picture. The winner uses more own techniques (`n_fe`) than its paired control in 7 of 11 competitions (the control more in 3, one tie), and paired `n_fe` medians are 6 for winners versus 3 for controls. Within a competition, winners tend to do *more* feature engineering, not less. The effect is modest and, at n = 11 pairs, not statistically significant (a sign test on 7-versus-3 gives $p \approx 0.34$); it is best read as a directional correction to the confounded aggregate rather than a firm result.

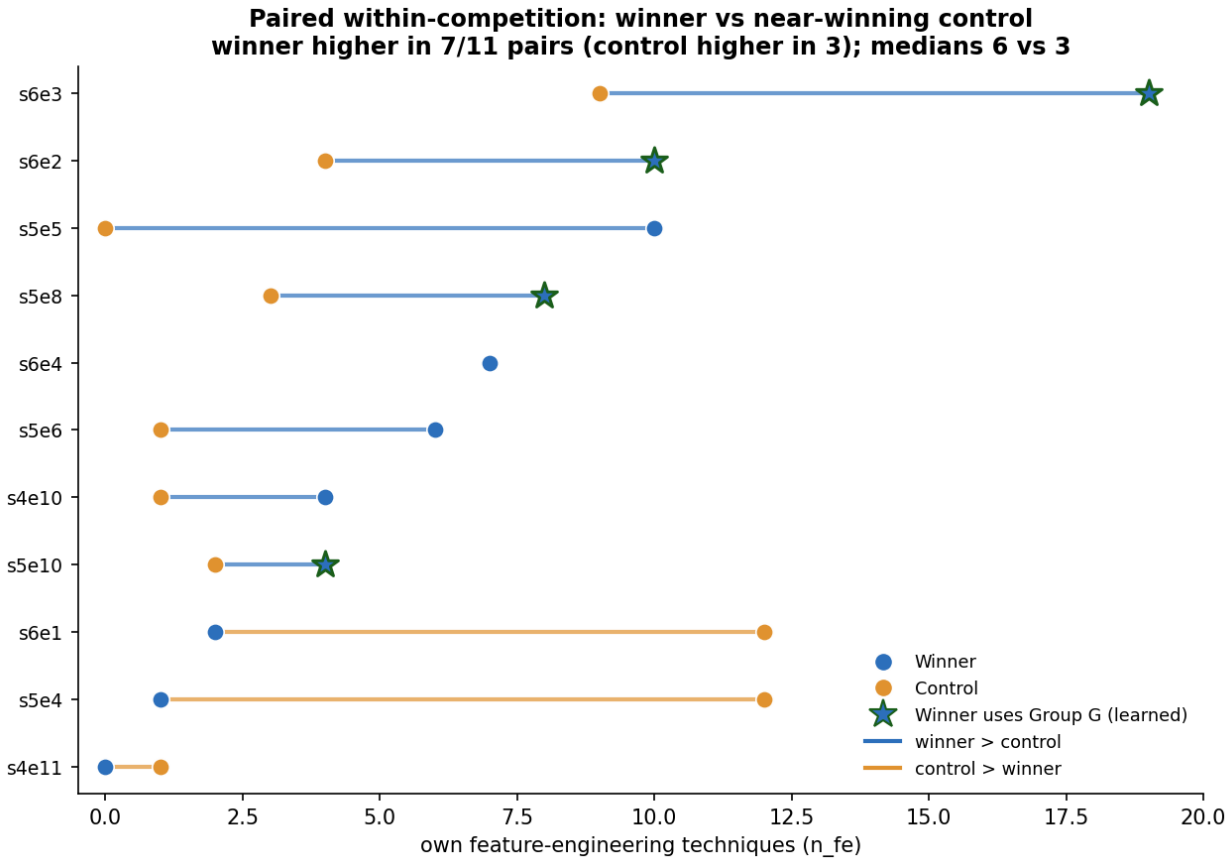
The reversals are paradigm-specific. The competitions where the control out-engineered the winner are interpretable rather than random: in two of them the winner won through a deliberately FE-light paradigm — a single 1,552-feature model trained on a pre-built external feature set (s5e4),

and a meta-learner stacking 190 out-of-fold predictions (s6e1) — while the control was an FE-intensive ensemble. This is consistent with Section 4.4: most winners do little feature engineering, and some win precisely by substituting model and ensemble craft for it.

Learned features are the one consistent signal. Across both framings, the only family that consistently favors the winner is learned/advanced features (Group G — autoencoder latents, genetic-programming features, PCA/SVD components). In the paired comparison, the winner uses a Group G technique and the control does not in 3 competitions, the reverse in 0, and both in 1 — the lone "both" case being a competition whose control is himself a genetic-programming specialist (an author signature; Section 4.6). Group G favors winners and never favors controls; at these counts (7 of 45 winners, 1 of 11 controls) this too is directional rather than significant. What most cleanly separates a winning tabular solution from a near-winning one is therefore less the *quantity* of feature engineering than the presence of learned, derived features that are harder to copy from a public notebook.

Scope. Both framings are bounded by the small sample and the documentation-self-selection limitation (Section 3.8). Because controls were selected partly for having writeups detailed enough to code, the control side is biased toward *more* documented feature engineering — which works against the paired finding rather than for it: the within-competition winner advantage emerges despite a control pool selected for FE detail. The comparison characterizes patterns among documented near-winners, not among typical competitors.





4.6 The Documented Top Is a Recurring Guild

The winner–control comparison (Section 4.5) found that feature-engineering volume distinguishes winners from near-winners only weakly and inconsistently. The composition of the corpus points to a complementary structural answer to what does: the documented top of these leaderboards is a small, recurring pool of highly skilled, socially connected competitors.

Author recurrence. Of the 11 near-winning controls, **5 share an author with an entry in the 45-winner corpus** — a 45% overlap. The recurring names include a multiple playground winner who is also a cited community hub and who additionally appears as a rank-25 control, and four other competitors who appear as both winners (in one competition) and near-winners (in another): a genetic-programming specialist, a stacking-ensemble specialist, a permutation-importance specialist, and a high-volume "diverse-blend" competitor. Because both the winners and the controls are documented top finishers, this overlap indicates that placing first versus placing fourth-to-twenty-fifth in a given month is, in part, an outcome drawn from the same small pool — the "winner" label captures a single month's result within a recurring elite rather than a categorically distinct group.

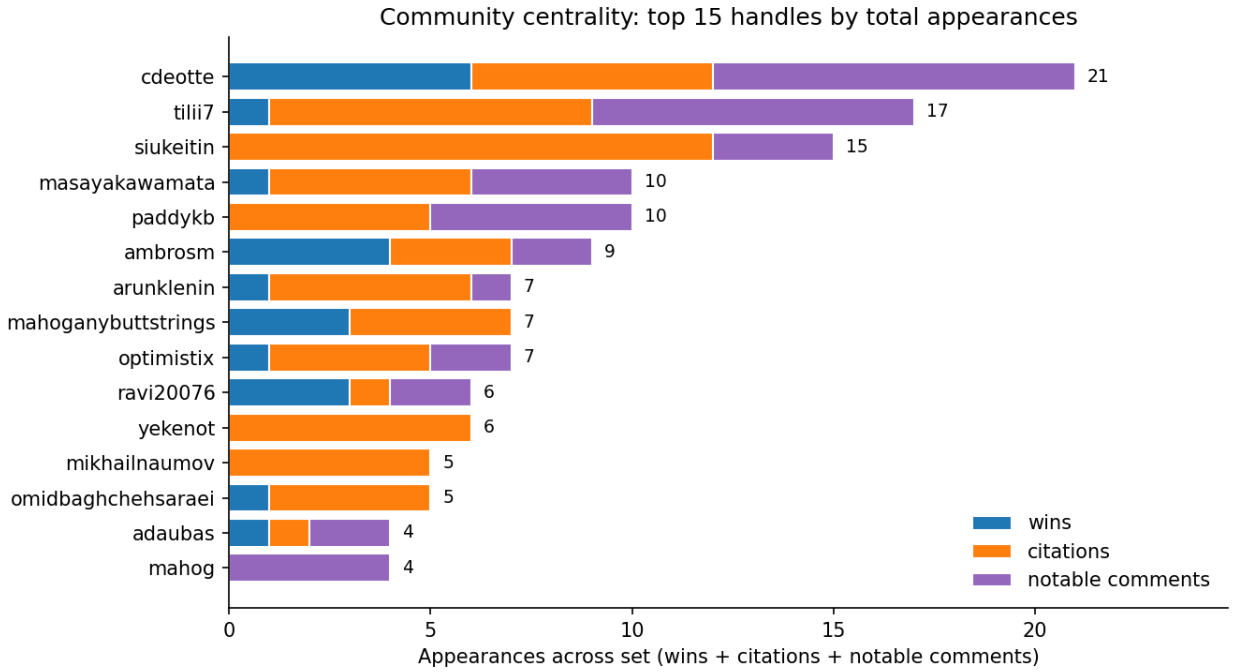
Part of this overlap is induced by the sampling instrument. Because controls were required to have published documentation (Section 3.8), and documentation at rank 4–25 comes disproportionately from a small set of habitual documenters, any documentation-filtered near-winner sample will over-represent the recurring elite — so the 45% figure is partly an artifact of the shared documentation filter rather than a direct measure of how concentrated *winning* is. What the filter does not explain

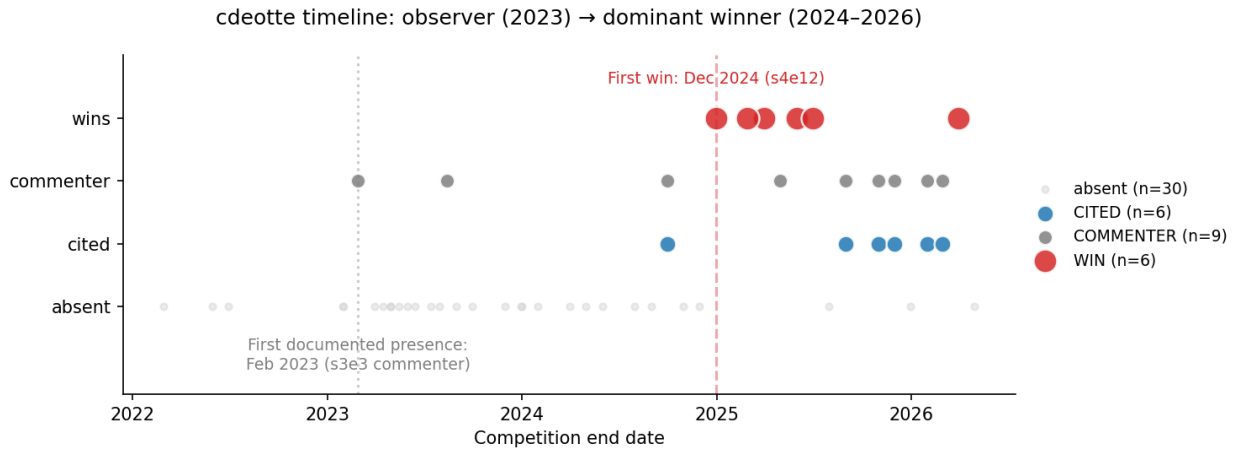
is why these recurring documenters keep finishing near the top rather than mid-field. Winning authorship is itself already concentrated: the 45 winner entries were authored by only **34 distinct competitors**, with five accounting for 16 of the entries and one alone for six — and the controls who overlap the winners are precisely these most-recurring names (the six-time author among them). The recurrence of *high finishes* within the documented pool is thus a genuine residual, observed through the documentation keyhole; the guild claim is scoped accordingly to the documented top, not to the entire competitor population.

Centrality structure. Aggregating citations across the corpus, three centrality roles appear (Figure 7). A handful of *pure technique-sources* — most prominently `siukeitin` (the single most-cited handle, appearing in 12 writeups), with `yekenot` (6), `paddykb` (5), and `mikhailnaumov` (5) further examples — supply techniques that propagate widely through winning solutions yet never themselves place top-three in the corpus; they are, in effect, the guild's armorers. A second tier *occasionally wins while being heavily cited* (`tilii7`: one win, eight citations; `masayakawamata`: one win, five). A third tier are *dominant competitors who are also cited sources* (`cdeotte`: six wins, six citations; `mahoganybuttstrings`: three wins; `AmbrosM` and partner: four wins).

Trajectory and propagation. The trajectory of the most central figure illustrates how membership forms (Figure 8): `cdeotte` first appears in February 2023 merely commenting on another competitor's winning writeup, then spends nearly two years as a commenter, cited source, and non-winning entrant (placing 81st and 806th in two competitions) before accelerating, from December 2024, into six wins in fifteen months. Technique knowledge moves through these hubs — canonized techniques show measurable adoption across the corpus, and in the most recent season an explicitly named LLM-assisted "playbook" methodology propagated from a single entry into the broader workflow (Section 4.7).

What this establishes. What the guild result establishes, robustly, is that the population from which documented winning solutions are drawn is small and recurring — and that this, more than any difference in feature-engineering inventory, is what the "winner" label tracks.





4.7 Paradigm by Era

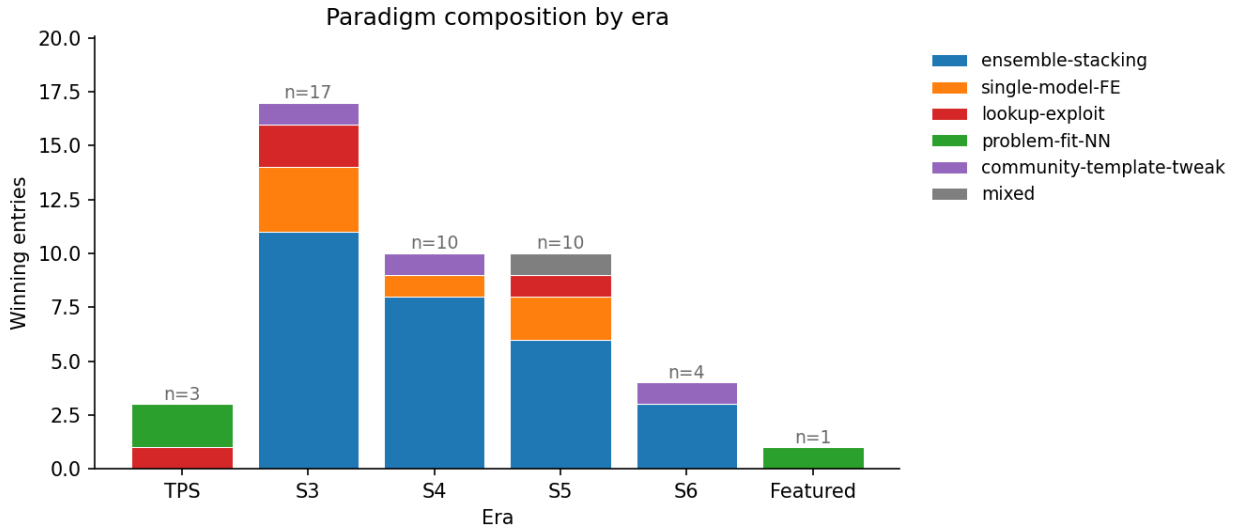


Figure 9. Paradigm composition by era.

Paradigm composition is era-dependent. The three Tabular Playground Series competitions from 2022 contributed two problem-fit-NN wins and one lookup-exploit win, with no ensemble-stacking wins. From PS Season 3 onward, ensemble-stacking is the modal paradigm in every era, accounting for 11 of 17 wins in S3 (65%), 8 of 10 in S4 (80%), 6 of 10 in S5 (60%), and 3 of 4 in S6 (75%). The only Featured competition in the corpus (ICR) is a problem-fit-NN win.

The single-model heavy-FE paradigm appears in three S3 entries, one S4 entry, and two S5 entries — distributed across eras but always representing a minority. The four lookup-exploit wins are distributed across eras (TPS, S3 ×2, S5) with no clear temporal trend, suggesting that lookup-exploit is competition-conditional (it requires an exploitable generator property) rather than era-conditional. The two residual categories are spread thinly: community-template-tweak appears once each in S3, S4, and S6, and the single mixed entry falls in S5.

The Season-6 shift. The four most recent entries (Playground Series Season 6, 2026) — three ensemble-stacking wins and one community-template-tweak — display a qualitative change in com-

petitive practice not seen in earlier eras. The Season-6 ensembles grew markedly larger — several blend 100 or more base models — and, for the first time in the corpus, competitors describe explicit use of large-language-model coding agents in their workflow, most prominently one entry's named "KGMON Playbook" methodology. This shift, read together with the migration of the feature-level edge toward learned representations (Section 4.5), suggests that the competitive margin is moving from hand-crafted features and modest ensembles toward very large, partly LLM-assisted pipelines whose components are expensive to reproduce. With only four Season-6 entries this is a directional observation rather than an established trend, but it is the most likely respect in which the four-paradigm typology will require extension as the platform evolves.

4.8 Constraint Cross-Tabulations

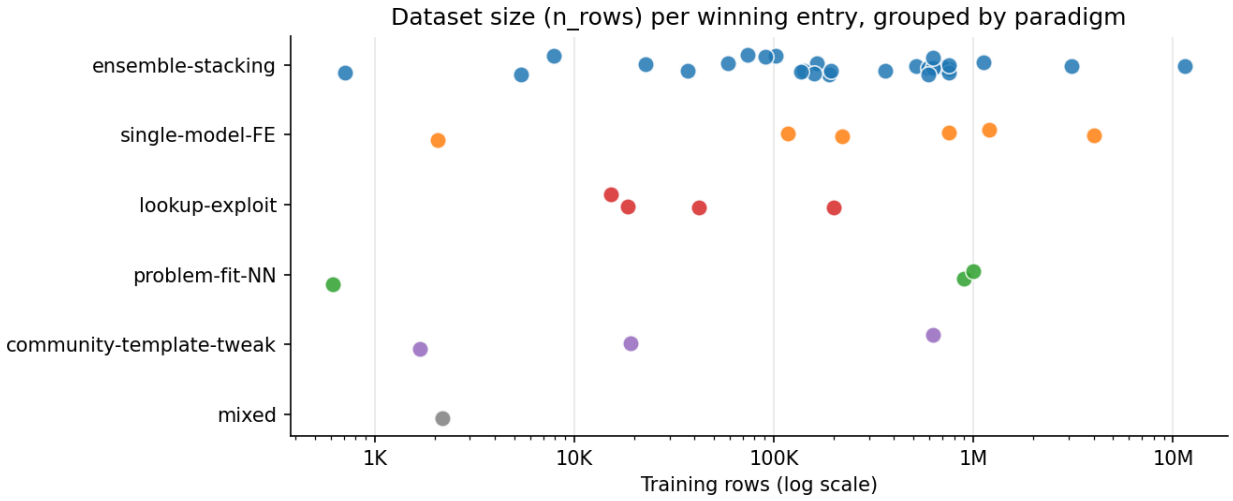


Figure 10. Dataset size (n_rows) by paradigm, log scale.

Three additional cross-tabulations characterize paradigm-versus-constraint structure.

Dataset size and paradigm. Lookup-exploit wins cluster at small-to-medium scale (15K–200K rows), with no lookup-exploit win above 200K rows in the corpus. Problem-fit-NN wins span the full range (617 rows to 1M rows). Single-model heavy-FE wins also span a wide range (2K to 4M rows). Ensemble-stacking wins occur at every scale (707 rows to 11.5M rows); its wider observed range partly reflects its far larger sample (n = 28 versus 3–6 for the other paradigms), but the dominant paradigm is clearly not scale-restricted.

External original and paradigm (Figure 11). All three problem-fit-NN wins occur in competitions where no external original is available — consistent with the interpretation that NN-primary strategies are reached for when the "exploit-original" toolkit is empty. Lookup-exploit wins by definition require an external original. The single columns-only use-mode entry in the corpus is one author's single-model heavy-FE win, confirming that the "use as columns" pattern is author-idiosyncratic rather than community-wide.

Photo-finish margin and paradigm. Using a metric-aware threshold (top-3 margin < 0.0005 absolute for ranking-style metrics, $< 0.1\%$ relative for regression metrics), 27 of 45 entries are classified as photo-finish wins. The photo-finish rate differs by paradigm (on small per-paradigm samples): community-template-tweak 100% (3/3), ensemble-stacking 68% (19/28), problem-fit-NN

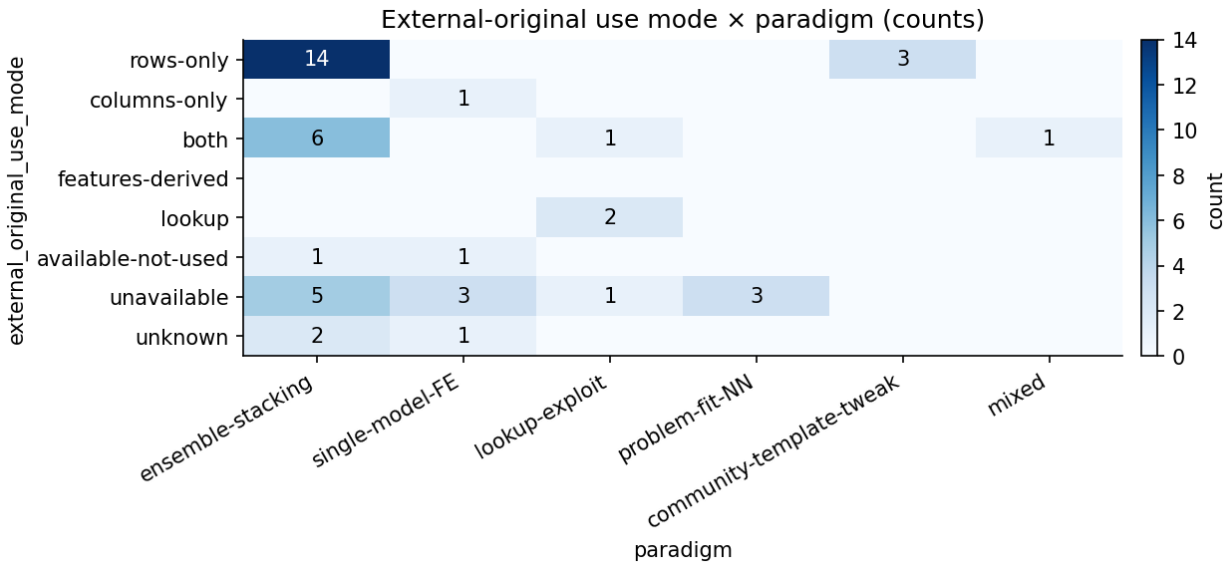


Figure 11. External-original use mode by paradigm.

67% (2/3), single-model heavy-FE 33% (2/6), lookup-exploit 25% (1/4). The lookup-exploit and single-model heavy-FE paradigms tend to win by wider margins, consistent with the interpretation that these paradigms change what is possible in the competition rather than squeezing the final 0.0001 from a saturated ensemble.

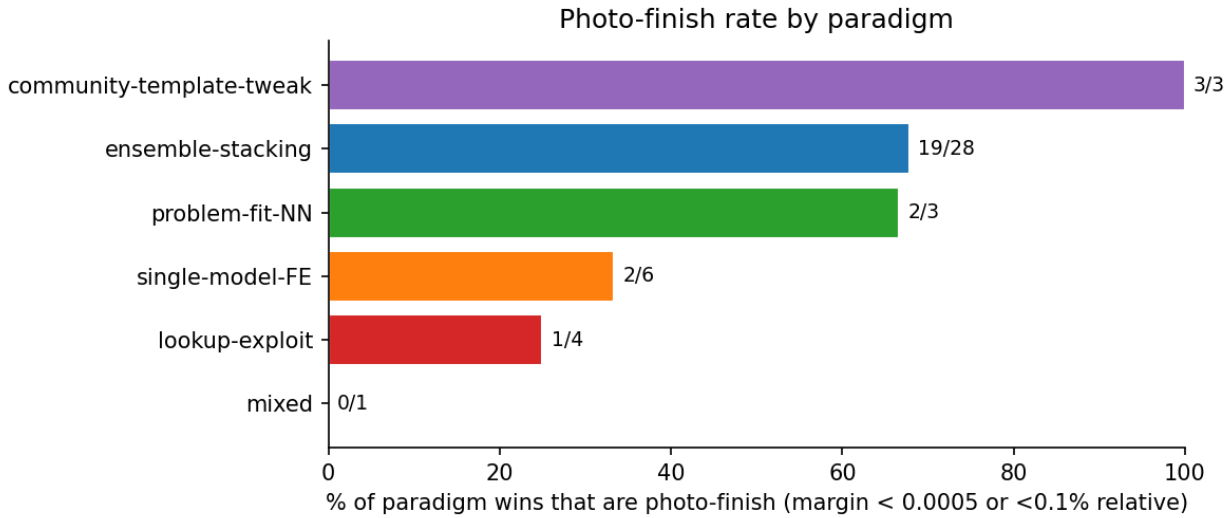


Figure 12. Photo-finish rate by paradigm.

4.9 Corpus Characteristics

For completeness, this section reports the descriptive composition of the winners corpus — the Pass 1 frequency distributions across model families, ensembling, cross-validation, and encoding — together with two attribution figures that underpin the community findings of Section 4.6. The Pass 1 schema captures *what techniques were used* in each solution (as distinct from what was

originated, which is captured in Pass 2).

Sample composition. The 45-entry corpus spans six competition cohorts: PS S3 (n = 17), PS S4 (n = 10), PS S5 (n = 10), PS S6 (n = 4), TPS (n = 3), and Featured (n = 1). By task type, 47% were binary classification (n = 21), 38% were regression (n = 17), and 16% were multiclass classification (n = 7). Sixty-seven percent of entries came from 1st-place solutions (30/45); 20% from 2nd-place (9/45); and 13% from 3rd-place (6/45).

Dataset characteristics. Training set size ranged from 617 to 11.5 million rows (median = 188,533; mean = 722,596). Feature counts ranged from 7 to 286 (median = 16). The large maximum feature count (286) came from one TPS entry. Excluding that outlier, 75% of entries had 21 or fewer features. Eighty percent of entries (36/45) had zero missing values — a structural property of synthetic Playground Series data — and only 9 entries had any missing values at all.

Dominant model family. Gradient-boosted machines (GBMs — XGBoost, LightGBM, or CatBoost) were the dominant base-model family in 35 of 45 entries (78%). Neural networks were the dominant base-model family in 7 entries (16%). One entry used a linear model (the GAM-based pulsar-detection solution at S3E10) as the primary approach, and two used ensemble-only architectures categorized as "other." For the blended ensemble-stacking entries that mix gradient-boosted and neural base models, this "dominant family" assignment is a judgment call; the count should be read as approximate at its margin.

Ensembling. Forty of 45 entries (89%) used an ensemble of multiple models. Among the 40 that ensembled, stacking was the most frequent method (mentioned in 21 entries), followed by weighted blending (9 entries), hill climbing (9 entries), and mean blending (8 entries). Many entries used multiple ensembling approaches simultaneously.

Cross-validation strategy. Among the 39 entries with a documented CV strategy, stratified k-fold was most common overall (19/39 = 49%), followed by plain k-fold (10/39 = 26%), repeated k-fold (5/39 = 13%), and repeated stratified k-fold (3/39 = 8%). CV strategy split by task type: among 21 binary classification entries, 14 (67%) used stratified k-fold; among 17 regression entries, 8 (47%) used plain k-fold and 4 (24%) used repeated k-fold. Three regression entries explicitly used stratified CV with target binning.

Encoding and external original use. Target encoding was the most common documented encoding (16 entries), followed by label encoding (6) and one-hot encoding (6). Thirty of 45 entries (67%) had an external original dataset available; the dominant use-mode when available is rows-only concatenation (17 entries), followed by both rows and columns (8 entries), with available-not-used (2 entries) and columns-only (1 entry) representing small categories.

Attribution structure. Two further figures underpin the community analysis of Section 4.6: Figure 13 shows the adoption frequency of canonized techniques across the corpus, and Figure 14 relates each entry's `origination_score` to the number of citations its author receives — quantifying the originator-versus-canonizer gap discussed there.

(Three findings originally reported as "unexpected" — the absence of pure forks among winners, the most-cited contributor never winning, and the contradiction of the "use-as-columns" folklore — are now integrated into Sections 4.2, 4.6, and 4.3 respectively.)

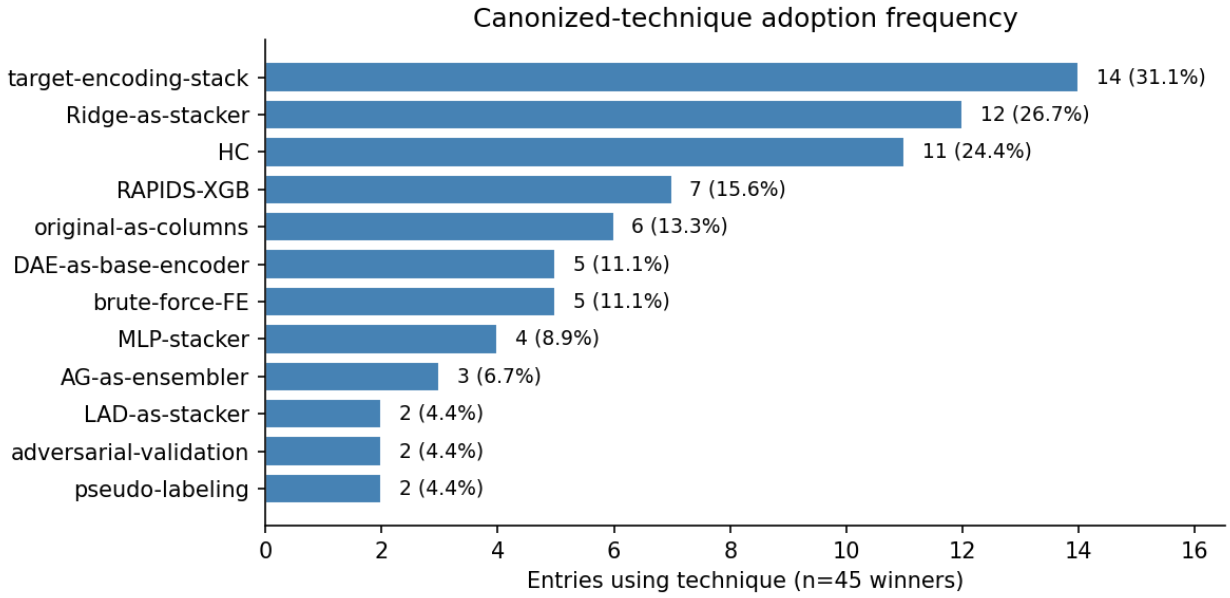


Figure 13. Canonized-technique adoption frequency.

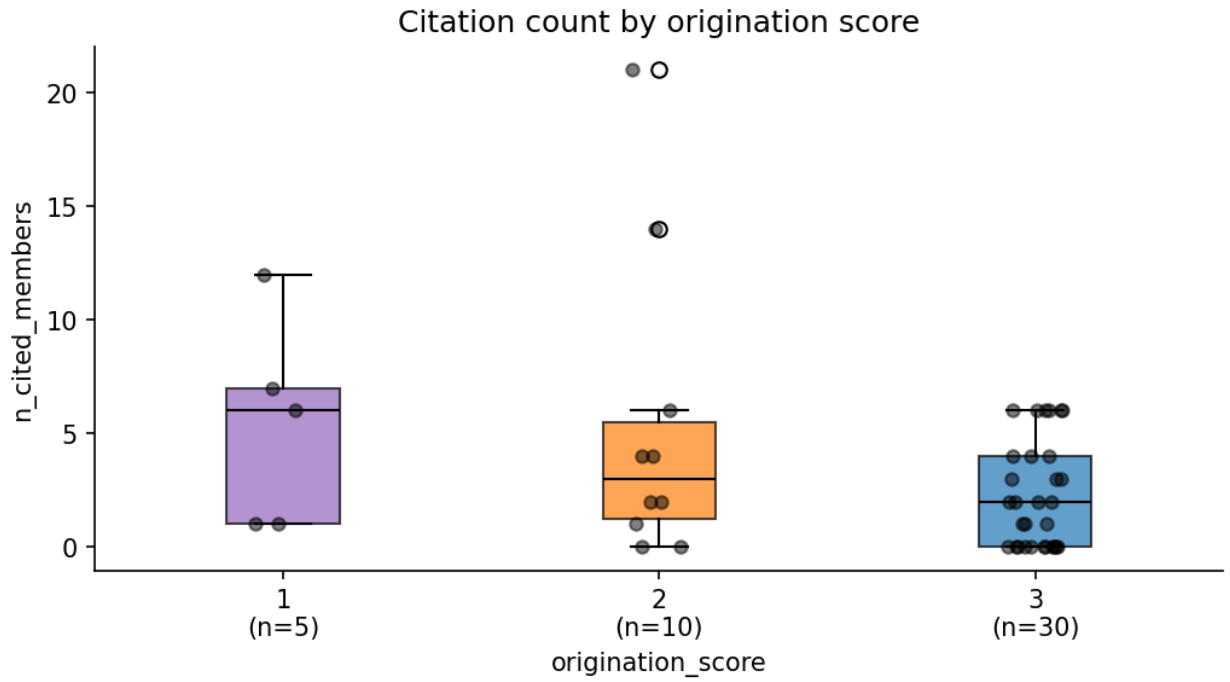


Figure 14. Citation count by origination score.

5. Discussion

5.1 Winning is not what the feature-engineering folklore predicts

The prevailing folklore of tabular machine learning holds that feature engineering is the decisive craft and that the strongest competitors win by out-engineering the field. The corpus contradicts the simple form of this claim in two ways: the median winner applies only a handful of their own catalogued feature-engineering techniques (four among well-documented entries; Section 4.4), and feature-engineering volume does not cleanly separate winners from near-winning controls — an aggregate comparison even favors the controls, though a within-competition paired comparison reverses it (Section 4.5). The most parsimonious interpretation is that the techniques that *matter* for winning tabular competitions have already been canonized and diffused through the community (Section 4.6), so that the high-leverage core — target encoding and its leak-safe within-fold form, a few interaction and encoding moves, and disciplined cross-validation — is shared widely enough across the documented elite that it no longer separates winners from near-winners, even though no single technique is universal (target encoding, the most common, reaches only ~40% of winners; Section 4.4). A technique that winners and near-winners are equally likely to use cannot distinguish first place from fourth; what remains to separate them is paradigm fit, ensemble craft, and the small residual edges examined below. This reading is consistent with the broader knowledge-propagation literature (Section 2.3): in an open community with fast, public diffusion, today's edge becomes tomorrow's baseline.

5.2 Learned features are the one feature-level edge that resists commoditization

The one feature-engineering family that favors winners over near-winners across every era is learned/advanced features — autoencoder latents, genetic-programming features, and dimensionality-reduction components — with the gap concentrated in heavyweight ensemble entries (Section 4.5). On thin counts (7 of 45 winners, 1 of 11 controls) this is directional rather than significant. Unlike the hand-crafted core, learned features do not commoditize in the same way: they are costlier to produce and harder to copy from a public notebook, which is consistent with their remaining a winner-favoring signal even as simpler techniques diffuse. The corpus shows a *persistent* edge, not an *emerging* one, however: Group G usage is roughly flat across eras in absolute terms (two winners in S3, one in S4, two in S5, two in S6), and the higher Season-6 proportion (two of four winners) rests on only four entries. Whether learned representations become *more* central as hand-crafted features commoditize — a direction that would cohere with the Season-6 move toward very large, expensive-to-reproduce ensembles (Section 4.7) — is plausible but not established by this corpus.

5.3 Winning is, in part, a community phenomenon

The recurring-guild finding (Section 4.6) reframes the question from *what winners do* to *who wins*. Even after discounting the portion of the winner–control author overlap that the shared documentation filter induces, a genuine residual remains: the documented top of these leaderboards is drawn from a small, recurring, socially connected pool, with individual competitors observed moving from commenter to cited source to dominant winner over roughly two years. This does not diminish the technical findings; it contextualizes them. If the differentiating techniques are largely shared (Section 5.1), then membership in the community that originates and canonizes those techniques — together with the skill and persistence that membership reflects — is itself part of what the "documented winner" label tracks.

5.4 What these findings do and do not establish

The strength of these conclusions rests on an honest boundary (Section 3.8). Because inclusion required a published writeup, and writeup publication is concentrated among habitual, brand-conscious competitors, *both* the winners and the near-winning controls are drawn from the **documented elite** of these competitions. The winner–control comparison therefore characterizes feature-engineering differences only between winners and **documented near-winners**, and the guild finding establishes that the **documented** top is small and recurring. Neither result speaks directly to the typical competitor who places mid-field and never publishes; a competitor of that kind is invisible to a writeup-based corpus by construction. The findings are best read as a characterization of how the documented top of tabular leaderboards is composed and behaves, not as a population-level account of all competitors.

A second boundary concerns the data regime. The corpus is overwhelmingly Playground Series — synthetic data generated from public real-world datasets — with a single Featured (real-data) entry. Several patterns reported here are partly artifacts of that regime: the prevalence of target encoding, the very existence of the lookup-exploit paradigm, and the heavy use of the external "original" dataset all depend on properties of synthetic generation (a known generator, recoverable feature–target pairs, an available source dataset) that do not hold in most real-world tabular problems or in proprietary Featured competitions. The findings most likely to transfer are those about *competition and community dynamics* rather than data properties — above all the recurring-guild composition, which depends on how competitors behave, not on how the data was generated. The typology and the small-core-plus-tail shape are only partly transferable: the lookup-exploit paradigm and the prevalence of target encoding are themselves synthetic-generation artifacts (as noted above) and would not survive a shift to real-world data. Generalization beyond synthetic-data competition practice is therefore not established by this corpus (Featured n = 1) and should be treated as an open question.

5.5 Implications and future work

For practitioners, the corpus suggests that returns to expanding feature-engineering *breadth* are limited: mastering the small canonized core (within-fold target encoding, sound cross-validation, and — above all — strong ensembling, used by 89% of winners) appears to matter more than accumulating bespoke features, and the remaining feature-level edge lies in learned representations rather than additional hand-crafted ones. Paradigm *choice* matters too, but — beyond a few competition-specific viability conditions (lookup-exploit only where the generator is exploitable; problem-fit neural networks where the data has exploitable structure; Section 4.2) — the corpus found no reliable rule mapping dataset constraints to a winning paradigm (Section 4.3), so "match the method to the data" is weaker advice than the folklore suggests. **For competition-as-method research** (Section 2.2), the results argue that the value extracted from competition writeups should be read with attention to community structure and technique diffusion, not only to technique inventories — and that near-winner controls are a cheap, informative addition to any winners-only corpus. **Methodologically**, the 53-technique taxonomy, reliability-checked with substantial family-level agreement (Section 3.6), is offered as a reusable instrument for coding tabular feature engineering, and the documentation-self-selection effect surfaced here is offered as a caution for any study that mines public competition writeups: such corpora describe the documented elite, and that filter must be stated rather than assumed away. **Future work** should attempt to escape the documentation filter directly — for example, by sampling public notebooks by leaderboard quartile or by coding solution descriptions from competitors who place mid-field and do not publish formal

writeups — which would convert the present winner-versus-near-winner comparison into a winner-versus-typical-competitor one.

Taken together, these results recast a familiar question. The decisive difference between a winning tabular solution and a near-winning one is, on this evidence, less the *quantity* of feature engineering than the combination of a well-canonized core, sound ensembling, a paradigm that fits the competition, and the harder-to-copy edges — learned representations, and the skill, persistence, and community membership that the recurring documented top reflects. What separates first place from fourth is, in short, as much about *who is competing* as about *what they engineer* — at least among the documented elite this corpus can see.

---

6. References

Bell, R. M., & Koren, Y. (2007). Lessons from the Netflix Prize challenge. *ACM SIGKDD Explorations Newsletter, 9*(2), 75–79. https://doi.org/10.1145/1345448.1345465

Borisov, V., Leemann, T., Seßler, K., Haug, J., Pawelczyk, M., & Kasneci, G. (2024). Deep neural networks and tabular data: A survey. *IEEE Transactions on Neural Networks and Learning Systems, 35*(6), 7499–7519. https://doi.org/10.1109/TNNLS.2022.3229161

Caruana, R., & Niculescu-Mizil, A. (2006). An empirical comparison of supervised learning algorithms. In *Proceedings of the 23rd International Conference on Machine Learning* (pp. 161–168). Association for Computing Machinery. https://doi.org/10.1145/1143844.1143865

Erickson, N., Mueller, J., Shirkov, A., Zhang, H., Larroy, P., Li, M., & Smola, A. (2020). *AutoGluon-Tabular: Robust and accurate AutoML for structured data.* arXiv. https://arxiv.org/abs/2003.06505

Gijsbers, P., Bueno, M. L. P., Coors, S., LeDell, E., Poirier, S., Thomas, J., Bischl, B., & Vanschoren, J. (2024). AMLB: An AutoML benchmark. *Journal of Machine Learning Research, 25*(101), 1–65. http://jmlr.org/papers/v25/22-0493.html

Gorishniy, Y., Rubachev, I., Khrulkov, V., & Babenko, A. (2021). Revisiting deep learning models for tabular data. In *Advances in Neural Information Processing Systems* (Vol. 34, pp. 18932–18943). Curran Associates. https://arxiv.org/abs/2106.11959

Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2022). M5 accuracy competition: Results, findings, and conclusions. *International Journal of Forecasting, 38*(4), 1346–1364. https://doi.org/10.1016/j.ijforecast.2021.11.013

Merton, R. K. (1968). The Matthew effect in science. *Science, 159*(3810), 56–63. https://doi.org/10.1126/science.1

von Krogh, G., & von Hippel, E. (2006). The promise of research on open source software. *Management Science, 52*(7), 975–983. https://doi.org/10.1287/mnsc.1060.0560

Zhang, T., Zhang, Z., Fan, Z., Luo, H., Liu, F., Liu, Q., Cao, W., & Li, J. (2023). OpenFE: Automated feature generation with expert-level performance. In *Proceedings of the 40th International Conference on Machine Learning* (PMLR Vol. 202, pp. 41880–41901). https://arxiv.org/abs/2211.12507

Note. The 45 winning Kaggle solutions and 11 near-winning controls (writeups and notebooks) that constitute the study's primary data are cited by competition URL in the per-writeup re-evaluation documents under `analysis/writeup-reevaluation/`, following the convention that primary-source online documents are referenced with the data rather than in the References list.